

Neural Networks - Teaching Computers to Think!

IITB Analytics Club Winter in Data Science, Guided Project

Vidit Nagpurkar

25B1270

Mentors: Singh Abhijeet Arbind, Vibhor Bhartiya

January 30, 2026

Contents

1	Linear and Polynomial Regression methods for data fitting	3
1.1	Custom Gradient Descent Implementation	3
1.2	Feature Engineering and Preprocessing	3
2	RNNs to Transformers	5
2.1	Recurrent Neural Networks (RNN) and LSTM	5
2.2	The Transformer Architecture	5
3	Research Analysis	6
3.1	The Architectural Shift: From RNNs to Transformers	6
3.2	DialoGPT	6
3.3	Concepts from Text Vectorization	7
3.4	Streamlit	7
4	Final Project: Movie Chatbot	7
4.1	Implementation	7
4.2	Challenges	7
4.3	Learnings	7
5	Comparative Persona Analysis	8
5.1	Test Case 1:	8
5.2	Test Case 2:	8
5.3	Test Case 3:	8
5.4	Test Case 4:	8
6	Conclusion	10
6.1	Summary	10
6.2	Learning Process	10
6.3	Future Scope	10
6.4	Final Remarks	10
7	References	11

1 Linear and Polynomial Regression methods for data fitting

1.1 Custom Gradient Descent Implementation

The primary focus in week-2 revolved around an implementation of gradient descent from scratch without high level abstractions or prebuilt libraries like scikit learn.

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 \quad (1)$$

Optimization was achieved via Vectorized Gradient Descent. Unlike iterative loops, vectorization utilizes NumPy's ability to process n-dimensional arrays simultaneously. Vectorised Gradient descent was used for optimisation purposes. Instead of using for loops, vectorisation uses NumPy and its ability to process large arrays quickly.

$$\theta := \theta - \alpha \frac{1}{m} X^T (X\theta - y) \quad (2)$$

1.2 Feature Engineering and Preprocessing

- **Target Encoding:** Implemented for the "City" variable to capture geographic trends in the model implemented for rent prices. City names like Mumbai, Bangalore, Hyderabad etc cannot be encoded directly using discretised values like one-hot encoding.
- **Z-Score Normalization:** Features like "House size" had greatly varying scales and orders of magnitude. This caused gradients to explode and vanish leading to highly random results from the model. To counter this, Z-score normalisation was implemented. All features in the data were scaled to a mean of 0 and a standard deviation of 1.

$$z = \frac{x - \mu}{\sigma} \quad (3)$$

- **Text Data Parsing:** Data available for features of "Furnishing status and "Floor level" were available in text and not useful for training the model despite being major factors determining rent in the real world. These features were included by engineering a custom parsing function that extracted floor number and total floors data from the irregular strings. Categorical labels like "Ground" and "Basement" were detected by the function and converted into the appropriate numerical quantities. This also allowed for the addition of a "Floor ratio" feature for additional data to the model for faster learning rates.

- **Fine-tuning Learning Rate:** Though the initial implementation was simple, the learning rate proved to be a difficult value to tune for a custom-made model. The value of 10^{-2} that worked for the fruit harvest data simply did not work for the rent model and a much finer learning rate was required. Making the learning rate too fine led to redundant steps during the process and a much longer training time. Finding the value that worked for larger data sets through trial and error was a slow process.

Solving these issues provided to me a deeper insight into why preprocessing is much more important than the model itself.

Data	Mean Absolute Percentage Error
Mango Harvests	1.2036%
Orange Harvests	0.8154%

2 RNNs to Transformers

While the eventual final project used the Transformer model, the predecessors to it were also explored to understand the current state of the art

2.1 Recurrent Neural Networks (RNN) and LSTM

The problem with RNN is the vanishing gradient problem where it isn't able to process large relations in movie dialogue.

- **LSTM (Long Short Term Memory):** Introduced gating mechanisms like Forget, Input and Output gates to maintain cell states.
- **Limitation:** The limitation is that sequential processing is still slow and has a very local understanding of the current sentence

2.2 The Transformer Architecture

The self-attention mechanism was used in the final project where every word in the prompt is able to interact with every other word simultaneously

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (4)$$

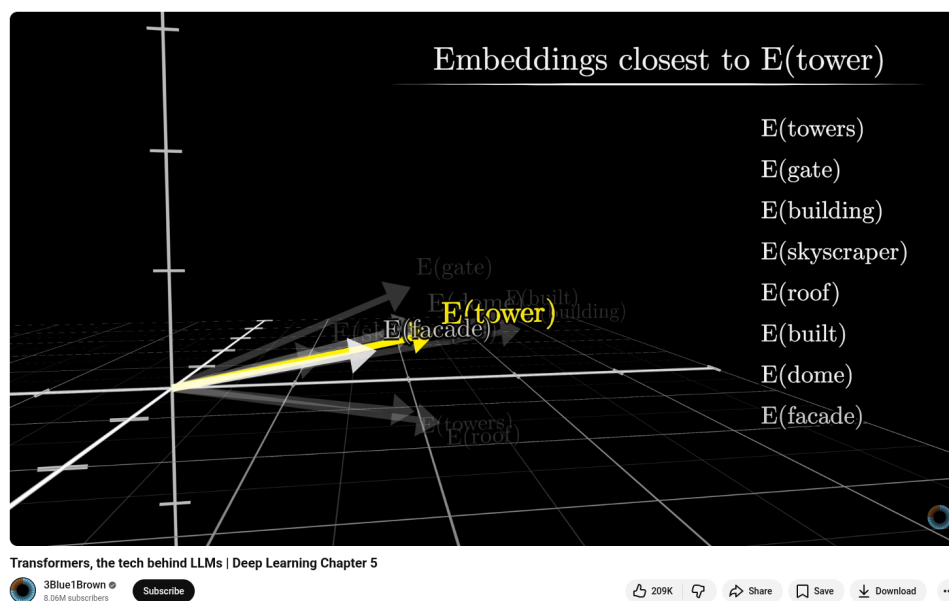


Figure 1: Transformers and Embedding, the fundamental technology behind DialoGPT

3 Research Analysis

Understanding the concepts behind Deep Learning and DialoGPT, which was used in the final project, the project shifted from simple neural networks to massive pre-trained models.

3.1 The Architectural Shift: From RNNs to Transformers

Before using DialoGPT, understanding the evolution of NLP was necessary

1. The Limitations of Recurrent Neural Networks: Early videos showed that RNNs process words as they see them, meaning they're only able to process the current word. I learned that this is like reading a book but you forget what happened at the start. By the time the model reaches the end of a sentence it doesn't know what was in the start. This made chatbots of the time feel like they were very repetitive and disconnected.

2. The Transformer Advantage: The "T" in GPT stands for Transformers. I learned that transformers use a method called "Self-Attention". Instead of processing words one at a time it looks at every word in the sentence simultaneously. This effectively allows to understand words in context through a kind of "global understanding". This is what makes modern models feel much more human than older chatbots

3.2 DialoGPT

[The base of the final project was DialoGPT, a large scale, open-domain and transformer based AI chatbot developed by Microsoft. It's based on the GPT-2 Architecture.

"DialoGPT: Large-Scale Generative Pre-training for Conversational Response Generation" by Zhang et al. (ACL 2020).

1. The Scale of Data: DialoGPT was trained on 147 million conversations between users on Reddit on comment threads between 2005 and 2017. This is what makes the model feel like it has common sense. It has seen millions of humans argue and joke on Reddit. Before the model training was done, it already knew how a conversation works.

2. Tunability A special aspect of DialoGPT, and why it was chosen for the project is that it is highly tunable. The researchers at Microsoft designed the model to be very flexible so that it could adapt different "personas". This is what I used and took their Reddit trained model while steering it towards the movie scripts.

3. Humanlike dialogue: The researchers found that DialoGPT attains performance close to human levels in single-turn dialogue. They used both automatic metrics and human evaluators to prove that DialoGPT produces responses that are: DialoGPT's conversations are very humanlike, especially in a single back and fourth. The conversations feel humanlike because of 3 major reasons

- **Relevant:** The bot actually answers the user's question.
- **Contentful:** It doesn't just say "I don't know" or "Okay", it provides interesting information.
- **Context-Consistent:** It maintains the vibe of the conversation.

3.3 Concepts from Text Vectorization

To understand how transformer maths works in actuality, I studied a concept known as Text Vectorisation.

- **TF-IDF and more:** I learned about TF-IDF which uses word frequencies as a major factor. This is not used in modern models anymore though. Models like DialoGPT use Embeddings.
- **Word Space:** In the model's brain words are stored as vectors in a multidimensional space which is very large. Words that have similar meanings are very close together (Dot product is large). This lets the model understand synonyms in text.

3.4 Streamlit

Streamlit was used to make the colab implementation into a usable app. I learned how to write basic streamlit apps for this purpose.

4 Final Project: Movie Chatbot

4.1 Implementation

A large dataset of movie conversations was used to train the DialoGPT model to shift its talking from generic conversations to more movie like dialogues. Trained model was integrated into a conversational UI deployed using Streamlit. .txt file containing conversations were parsed using python string manipulation and model was trained on Google Colab using a T4 GPU.

4.2 Challenges

Early iterations of the model generated movie script formatting like “———” “END” and “INTJ”. This happened because the raw data provided contained this formatting syntax that was included in the added context for the model. This was fixed by implementing a RegEx layer to clean the commonly observed faulty output phrases ensuring that the model learned dialogue instead of script structure. The model was observed to frequently get into loops where it repeated the same phrase again and again, possibly for dramatic effect. This was fixed by introducing a repetition penalty and adjusting the Temperature variable to 0.8. Repetitive patterns were observed to reduce significantly.

4.3 Learnings

Explored the evolution of NLP from RNN and LSTM to the transformer method and its self-attention mechanism which allows it to process context much more effectively. The project used BPE tokenization through DialoGPT but traditional methods like TF-IDF were also covered for vector representation of words. Learnt Transfer Learning workflow that allows to steer a pretrained model towards a specific persona using added context. Understood how to make an app interface using Streamlit. I implemented session state management so that the chatbot remembers the context from previous conversations.

Movie Chatbot



Greetings, Fellow Traveller, what brings you here



Just a little rain. It's supposed to come in early Sunday morning. I'm trying my best not too much!



Ah I hope you find your way



Oh God, the sound of it. The rain, it is so loud. Every drop hits me like an arrow. That must be painful for you...

Figure 2: Final implementation of the Streamlit UI showing cinematic responses.

5 Comparative Persona Analysis

5.1 Test Case 1:

User: "I thought I could trust you, why did you tell them?"

Regular Response: Because they could be lying.

Cinematic Response: Because you got your way with me.

5.2 Test Case 2:

User: "Who was that man? The one you were talking to"

Regular Response: He was the one that was standing behind the camera.

Cinematic Response: Well – somebody I could hear. dramatic pause (the double dash).

5.3 Test Case 3:

User: "Well, looks like this is the end, they've got us surrounded."

Regular Response: I'm sure they'll give us some sort of an incentive to keep going.

Cinematic Response: I'm going to go find my mother and let her know this situation.

5.4 Test Case 4:

User: "You killed my brother! You cowardly son of a gun! Gunned him down when he wasn't hardly looking."

Regular Response: That's just the way you talk. You're a real pleasure to be around, though. How are things?

Cinematic Response: He was a coward! He didn't even know the difference between the two words!

6 Conclusion

6.1 Summary

The project has acted as a comprehensive exploration of the transition in technology from statistical data fitting to more modern artificial intelligence. By beginning with the manual implementation of Vectorised Gradient Descent and Polynomial curve fitting I understood how machines optimise functions at a fundamental level. The challenges I faced in the earlier weeks provided practical lessons in data normalisation and feature scaling

The second part of the project showed the power of Transfer learning. Using the DialoGPT model as a foundation I understood how a model trained on social media conversations can be effectively steered towards a cinematic persona. The shift in the responses from the chatbot can clearly be observed after training it on the large dataset.

6.2 Learning Process

Other than the code itself, this project helped me to understand the importance of the lifecycle of data. The transformer architecture is very sophisticated but its performance is very dependent on the quality of preprocessing. The use of a RegEx cleaning layer to remove script artifacts was very important as far as the final user experience is concerned

6.3 Future Scope

Reinforcement Learning from Human Feedback (RLHF): The current model is very heavily reliant on supervised fine tuning and repetitive loops are a very big issue. Future implementations could punish this repetitive nature and hallucinations using RLHF that are still visible.

6.4 Final Remarks

In conclusion, this project has given me a very comprehensive toolkit for AI development in the future. Going from the calculus involved with cost functions to the very complicated Transformer based models that use self attention, I now have an appreciation for the potential of NLP. As the field goes forward, the techniques I learned in this project will remain foundational to any future work I pursue in Machine Learning.

7 References

1. Microsoft Research. (2019). *DialoGPT: Large-Scale Generative Pre-training for Conversational Response Generation*.
2. Vaswani et al. (2017). *Attention is All You Need*.
3. Colah's Blog. *Understanding LSTM Networks*.